

EC4219: Software Engineering

Lecture 15 — Let-Polymorphic Type System

Sunbeom So
2025 Spring

Motivation

- Our type system is useful but it is not as expressive as we would like it to be. In particular, it does not support **polymorphism**¹. For example, it rejects the following program:

```
let f = proc (x) x in
  if (f (iszero (0))) then (f 11) else (f 22)
```

- Polymorphic functions are widely used in practice, so OCaml supports polymorphism:

```
# let f = fun x -> x in
  if (f (0=0)) then (f 11) else (f 22);;
- : int = 11
```

- Let's extend our type system to the let-polymorphic type system, the ML-style polymorphism.

¹Polymorphism refers to the language mechanisms that allow a single part of a program to be used with different types in different contexts

What went wrong?

```
let f = proc (x) x in  
  if (f (iszero (0))) then (f 11) else (f 22)
```

- We assign type $t \rightarrow t$ to f , generating the constraint that the argument and return types are the same.
- Intuitively, the program can be well typed because the all usages of f satisfy the required constraint:
 - ▶ In $(f \text{ (iszero 0)})$, we can assign $\text{bool} \rightarrow \text{bool}$ to f .
 - ▶ In $(f \text{ 11})$ and $(f \text{ 22})$, we can assign $\text{int} \rightarrow \text{int}$ to f .
- However, our type checking algorithm uses the same type variable t in both cases and generates the spurious constraint that $\text{bool} = \text{int}$.
- Any idea to fix this problem?

A Simple Solution

Introduce a different variable t for each use of f . This is easily accomplished by substituting the body of f for each occurrence of f . For example, convert the program

```
let f = proc (x) x in
  if (f (iszero (0))) then (f 11) else (f 22)
```

into the following before type-checking:

```
if ((proc (x) x) (iszero (0)))
then ((proc (x) x) 11)
else ((proc (x) x) 22)
```

which is accepted by our type system as we can generate different type variables for different copies of the procedure.

Typing Rule

Instead of the previous typing rule for let:

$$\frac{\Gamma \vdash E_1 : t_1 \quad [x \mapsto t_1]\Gamma \vdash E_2 : t_2}{\Gamma \vdash \text{let } x = E_1 \text{ in } E_2 : t_2}$$

we use the new typing rule:

$$\frac{\Gamma \vdash [x \mapsto E_1]E_2 : t_2 \quad \Gamma \vdash E_1 : t_1}{\Gamma \vdash \text{let } x = E_1 \text{ in } E_2 : t_2}$$

Similarly, instead of:

$$\mathcal{V}(\Gamma, \text{let } x = e_1 \text{ in } e_2, t) = \\ \mathcal{V}(\Gamma, e_1, \alpha) \wedge \mathcal{V}([x \mapsto \alpha]\Gamma, e_2, t) \text{ (new } \alpha)$$

The corresponding algorithm for generating type equation:

$$\mathcal{V}(\Gamma, \text{let } x = e_1 \text{ in } e_2, t) = \mathcal{V}(\Gamma, e_1, \alpha) \wedge \mathcal{V}(\Gamma, [x \mapsto e_1]e_2, t) \text{ (new } \alpha)$$

The ordinary unification algorithm does the rest.

Flaw: Efficiency

The method is not efficient if the body of `let` contains many occurrences of the bound variables:

```
let a = <complex code> in
  let b = a + a in
    let c = b + b in
      let d = c + c in
        ...
```

The typing rule can cause the type-checker to perform an amount of work that is exponential in the size of the original code.

Example

For example, try to evaluate the following OCaml program. It takes a very long time to typecheck.

```
let f0 = fun x -> (x,x) in
  let f1 = fun y -> f0 (f0 y) in
    let f2 = fun y -> f1 (f1 y) in
      let f3 = fun y -> f2 (f2 y) in
        let f4 = fun y -> f3 (f3 y) in
          let f5 = fun y -> f4 (f4 y) in
            f5 (fun z -> z)
```

Summary

- We extended our type system (called *simple type system*) to **let-polymorphic type system**, the core of ML type system.
- The extension is conservative:

$$\Gamma \vdash_{simple} E : T \implies \Gamma \vdash_{poly} E : T$$

Let-polymorphic type system accepts all programs acceptable by the simple type system.